

6.1 Введение в ООП

- 6.1.1 Что такое объектно-ориентированное программирование?
- 6.1.2 Зачем вообще нужен ООП и какие задачи он решает?
- 6.1.3 Чем ООП отличается от процедурного подхода?
- 6.1.4 Что такое объект простыми словами?
- 6.1.5 Что такое класс простыми словами?
- 6.1.6 Чем класс отличается от объекта?
- 6.1.7 Что такое состояние и поведение объекта?
- 6.1.8 Что такое поля и методы объекта?
- 6.1.9 Как создать объект в Java? Что делает оператор new?
- 6.1.10 Что такое ссылка на объект?

6.2 Классы и объекты

- 6.2.1 Как описывается класс в Java?
- 6.2.2 Что можно хранить внутри класса?
- 6.2.3 Что такое экземпляр класса?
- 6.2.4 Что такое поля экземпляра?
- 6.2.5 Что такое методы экземпляра?
- 6.2.6 Как обращаться к полям и методам объекта?
- 6.2.7 Что такое this и зачем он нужен?
- 6.2.8 Можно ли внутри класса обращаться к своим полям и методам без this?
- 6.2.9 Когда this обязателен, а когда нет?

6.3. Конструкторы

- 6.3.1 Что такое конструктор?
- 6.3.2 Чем конструктор отличается от обычного метода?
- 6.3.3 Что делает конструктор по умолчанию?
- 6.3.4 Что такое конструктор без параметров и конструктор с параметрами?
- 6.3.5 Можно ли перегружать конструкторы?
- 6.3.6 Что такое вызов this() внутри конструктора?
- 6.3.7 Зачем инициализировать объект через конструктор? где нет?

6.4 Основные принципы ООП

- 6.5.1 Какие основные принципы ООП существуют?
- 6.5.2 Что такое инкапсуляция как принцип ООП?
- 6.5.3 Что такое наследование?
- 6.5.4 Что такое полиморфизм?
- 6.5.5 Что такое абстракция?
- 6.5.6 Как эти принципы работают вместе в реальном коде?

6.5. Инкапсуляция

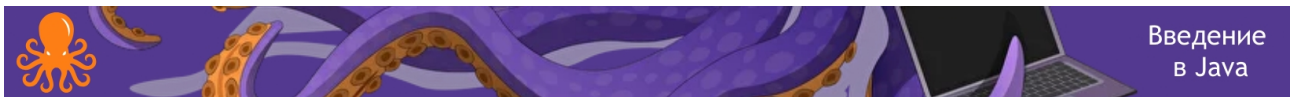
- 6.4.1 Что такое инкапсуляция?
- 6.4.2 Зачем скрывать внутреннее состояние объекта?
- 6.4.3 Что такое геттеры и сеттеры и зачем они нужны?
- 6.4.4 Почему не стоит делать все поля public?
- 6.4.5 Какие модификаторы доступа есть в Java?
- 6.4.6 Чем отличаются private, package-private, protected и public?
- 6.4.7 Когда сеттер лучше не делать вообще?
- 6.4.8 Как инкапсуляция помогает защищать объект от некорректного состояния?

6.6. Наследование

- 6.6.1 Что такое наследование в Java?
- 6.6.2 Зачем одному классу наследоваться от другого?
- 6.6.3 Что такое родительский и дочерний класс?
- 6.6.4 Что наследуется от родительского класса, а что нет?
- 6.6.5 Что такое extends?
- 6.6.6 Можно ли в Java наследоваться от нескольких классов?
- 6.6.7 Что такое super и зачем он нужен?
- 6.6.8 Как вызывать конструктор родительского класса?
- 6.6.9 Когда наследование полезно, а когда оно только вредит?

6.6. Полиморфизм

- 6.7.1 Что такое полиморфизм простыми словами?
- 6.7.2 Зачем нужен полиморфизм в Java?
- 6.7.3 Что такое статический полиморфизм?
- 6.7.4 Что такое перегрузка методов (overloading)?
- 6.7.5 По чему Java выбирает перегруженный метод?



- 6.7.6 Что такое динамический полиморфизм?
- 6.7.7 Что такое переопределение методов (overriding)?
- 6.7.8 Как Java определяет, какой переопределенный метод вызвать во время выполнения?
- 6.7.9 Чем перегрузка отличается от переопределения?
- 6.7.10 Что такое раннее и позднее связывание?
- 6.7.11 Почему переменная типа родителя может ссылаться на объект дочернего класса?
- 6.7.12 Что можно вызвать через ссылку родительского типа, а что нельзя?
- 6.7.13 Что такое upcasting и downcasting?
- 6.7.14 Зачем нужен instanceof?

6.8 Абстракция, абстрактные классы и интерфейсы

- 6.8.1 Что такое абстракция в ООП?
- 6.8.2 Зачем отделять “что делает” от “как делает”?
- 6.8.3 Что такое абстрактный класс?
- 6.8.4 Можно ли создать объект абстрактного класса?
- 6.8.5 Что такое абстрактный метод?
- 6.8.6 Что такое интерфейс в Java?
- 6.8.7 Чем интерфейс отличается от абстрактного класса?
- 6.8.8 Когда лучше использовать интерфейс, а когда абстрактный класс?
- 6.8.9 Что такое implements?
- 6.8.10 Можно ли реализовать несколько интерфейсов?
- 6.8.11 Что такое default-методы в интерфейсах?
- 6.8.12 Зачем в интерфейсах появились static- и private-методы?

6.9 Переопределение, перегрузка и работа методов

- 6.9.1 Какие правила есть у переопределения методов?
- 6.9.2 Можно ли переопределить private, static и final методы?
- 6.9.3 Что делает аннотация @Override?

6.9.4 Можно ли менять возвращаемый тип при переопределении?

6.9.5 Можно ли менять уровень доступа при переопределении?



6.9.6 Что происходит, если сигнатура метода меняется совсем немного?

6.9.7 Почему перегрузка определяется на этапе компиляции, а переопределение — во время выполнения?

6.10 Ключевые слова и особенности ООП в Java

6.10.1 Что такое static-поле и static-метод?

6.10.2 Чем static-члены отличаются от обычных полей и методов объекта?

6.10.3 Когда static уместен, а когда он ломает дизайн?

6.10.4 Что такое final-поле, final-метод и final-класс?

6.10.5 Зачем запрещать переопределение метода через final?

6.10.6 Что такое nested class и inner class на базовом уровне?

6.10.7 Что такое enum и почему его тоже можно рассматривать как часть ООП-модели?

6.11 Связи между объектами

6.11.1 Какие отношения между классами бывают кроме наследования?

6.11.2 Что такое ассоциация?

6.11.3 Что такое агрегация?

6.11.4 Что такое композиция?

6.11.5 Чем композиция отличается от наследования?

6.11.6 Почему во многих случаях композиция лучше наследования?

6.11.7 Что означает принцип “has-a” и чем он отличается от “is-a”?

6.12 Object и базовые методы объектов

- 6.12.1 Что такое класс Object и почему он важен?
- 6.12.2 Какие основные методы приходят от Object?
- 6.12.3 Что делает toString() и зачем его переопределять?
- 6.12.4 Что делает equals() и почему его нельзя путать с ==?
- 6.12.5 Что делает hashCode() и почему он связан с equals()?
- 6.12.6 Почему неправильная реализация equals/hashCode ломает коллекции?
- 6.12.7 Что делает getClass()?

6.13 SOLID

- 6.13.1 Что такое SOLID и зачем эти принципы нужны?
- 6.13.2 Что такое принцип единственной ответственности (SRP)?
- 6.13.3 Как понять, что у класса слишком много ответственности?
- 6.13.4 Что такое принцип открытости/закрытости (OCP)?
- 6.13.5 Как расширять поведение класса без изменения уже работающего кода?
- 6.13.6 Что такое принцип подстановки Барбары Лисков (LSP)?
- 6.13.7 Почему не любое наследование является корректным с точки зрения LSP?
- 6.13.8 Что такое принцип разделения интерфейсов (ISP)?
- 6.13.9 Почему лучше несколько маленьких интерфейсов, чем один “жирный”?
- 6.13.10 Что такое принцип инверсии зависимостей (DIP)?
- 6.13.11 Почему код лучше связывать с абстракциями, а не с конкретными реализациями?
- 6.13.12 Как SOLID помогает писать код, который проще тестировать, менять и расширять?

6.14 Частые ошибки и плохие практики в ООП

- 6.14.1 Почему наследование ради переиспользования кода может быть плохой идеей?
- 6.14.2 Почему слишком большие классы и “божественные объекты” — это проблема?
- 6.14.3 Почему большое количество сеттеров часто делает модель слабой?
- 6.14.4 Почему `static` везде подряд — это тревожный сигнал?
- 6.14.5 Почему нарушение инкапсуляции усложняет поддержку кода?
- 6.14.6 Почему глубокая иерархия наследования затрудняет понимание программы?

6.15 Продвинутые и полезные вопросы по ООП

- 6.15.1 В чем разница между “объектом как сущностью” и “объектом как структурой данных”?
- 6.15.2 Почему хорошая ООП-модель должна защищать инварианты объекта?
- 6.15.3 Что такое слабая связность и высокая связность внутри класса?
- 6.15.4 Почему композиция часто лучше наследования в реальном проекте?
- 6.15.5 Когда абстракция действительно полезна, а когда это лишний слой?
- 6.15.6 Как понять, что интерфейс выделен правильно?
- 6.15.7 Когда полиморфизм делает код чище, а когда только прячет простую логику?
- 6.15.8 Как ООП связано с тестируемостью и расширяемостью системы?